



udp UNIVERSIDAD
DIEGO PORTALES

UNIVERSIDAD DIEGO PORTALES
ESCUELA DE INFORMÁTICA &
TELECOMUNICACIONES

ESTRUCTURAS DE DATOS & ANÁLISIS DE ALGORITMOS

Laboratorio 4: Árboles de Búsqueda

Problema: Inventario del Club de Robótica

Integrantes:

Renato Parra

Matías Herrera

Profesor: Marcos Fantóval

Junio 2026

https://github.com/mvtiasfelipe69/Lab4_S5_MatiasHerrera_RenatoParra

Índice

1. Introducción	2
2. Etapa 1: Implementación	3
2.1. Clases del programa	3
2.2. Clases de Princeton utilizadas	3
2.3. Descripción detallada de las clases	4
2.4. Generador de Operaciones y Decisiones de Diseño	5
3. Etapas 2 y 3: Análisis Empírico	6
3.1. Estructura del software experimental	6
3.2. Desafíos enfrentados	6
3.3. Mediciones empíricas	6
4. Etapa 4: Análisis Teórico Big-O	9
4.1. Tabla de símbolos	9
4.2. BST — Caso promedio $O(\log n)$, Peor caso $O(n)$	9
4.3. RedBlackBST — Balanceo $O(\log n)$ garantizado	9
5. Etapa 5: Comparación Teoría vs. Práctica	10
6. Conclusiones	11
Referencias	12

1. Introducción

En este laboratorio se implementó un sistema de inventario simplificado para el Club de Robótica de la Universidad Diego Portales. El objetivo es comparar empíricamente el rendimiento de dos estructuras de datos: BST y RedBlackBST, ambas provenientes de la biblioteca `algs4` del texto guía de Sedgewick y Wayne.

El Club de Robótica administra una gran variedad de componentes electrónicos y mecánicos. Cada semana se registran compras de nuevos materiales, préstamos a estudiantes, devoluciones de componentes y bajas por daño u obsolescencia. El problema principal a resolver en este informe es: ¿Qué estructura de datos es más conveniente para administrar un inventario dinámico sometido a una larga secuencia de compras, consultas, préstamos, devoluciones y bajas?

Para responder a esto, se abstraigo el problema utilizando el concepto de Tablas de Símbolos (*Symbol Tables*) implementando dos soluciones basadas en árboles: un Árbol Binario de Búsqueda tradicional (BST) y un árbol auto-balanceado (RedBlackBST). Mediante simulaciones de carga intensiva, se midieron los tiempos de CPU y la estructura final (altura del árbol) para contrastar los resultados de la práctica con el análisis teórico Big-O.

Las estructuras comparadas son:

- **BST<Key, Value>:** Árbol binario de búsqueda estándar, no balanceado. Su forma geométrica depende exclusivamente del orden en el que se insertan los datos.
- **RedBlackBST<Key, Value>:** Árbol rojo-negro que mantiene su nivel de balance mediante invariantes estrictas, lo que garantiza una altura máxima acotada a $2 \log_2 N$ en el peor de los casos.

Los entregables asociados a este informe técnico son:

- **Código fuente:** Clases en Java con la implementación de los adaptadores y simulaciones.
- **Archivos CSV:** Las métricas extraídas del experimento (60 iteraciones para cada tamaño m).
- **Planilla Excel:** El archivo `analysis.xlsx` con los cálculos estadísticos, histogramas y gráficos correspondientes.

2. Etapa 1: Implementación

2.1. Clases del programa

El sistema se diseñó separando la lógica del negocio de las estructuras de datos, aplicando Tipos de Datos Abstractos (TDA). El simulador está compuesto por las siguientes clases y artefactos en Java:

- **InventoryItem**: Modela un componente físico del inventario. Administra atributos como `id`, `name`, `category`, `location`, y diferentes niveles de stock. Incluye los métodos de mutación que aplican las reglas del negocio.
- **InventoryOperation**: Encapsula una transacción particular. Contiene el tipo de operación, la clave del componente, la cantidad y un objeto **InventoryItem** si se trata de un ingreso nuevo.
- **OperationType**: Enumerador con cinco operaciones semánticas: `PURCHASE`, `QUERY`, `LEND`, `RECEIVE` y `DISPOSE`.
- **InventoryIndex**: Interfaz que funciona como contrato común (patrón *Strategy/Adapter*) y unifica las operaciones de la tabla de símbolos (`put`, `get`, `delete`).
- **BSTInventoryIndex** y **RedBlackInventoryIndex**: Clases que implementan **InventoryIndex** como adaptadores, delegando las llamadas lógicas directamente a las estructuras de Princeton.
- **DataGenerator**: Se encarga de generar listas de operaciones aleatorias y reproducibles aprovechando la librería `algs4`.
- **Experiment**: Clase principal. Coordina todo el experimento, iterando sobre los tamaños, ejecutando las transacciones, midiendo el tiempo y generando los archivos CSV.

2.2. Clases de Princeton utilizadas

Para el desarrollo del código se aprovecharon las herramientas de la librería `algs4`:

- **BST**: Tabla de símbolos basada en un árbol binario de búsqueda clásico.
- **RedBlackBST**: Tabla de símbolos basada en un árbol rojo-negro.
- **StdRandom**: Usado para generar datos reproducibles fijando la semilla con `setSeed`.
- **StopwatchCPU**: Utilizado para cronometrar ciclos de CPU, evitando que procesos en segundo plano del reloj del sistema afecten la medición.
- **Out**: Para escribir los resultados del experimento en formato CSV.

De acuerdo con las instrucciones de la cátedra, las lógicas de rotación e inserción de los árboles no se programaron desde cero, sino que se utilizaron directamente las provistas por la librería oficial.

2.3. Descripción detallada de las clases

A continuación, se presentan fragmentos relevantes de la implementación, mostrando cómo se protegió la integridad de los datos en el inventario.

```
1 public void addStock(int quantity) {
2     if (quantity <= 0) return;
3     stockTotal += quantity;
4     stockAvailable += quantity;
5 }
6
7 public boolean lend(int quantity) {
8     if (quantity <= 0 || quantity > stockAvailable) return
9         false;
10    stockAvailable -= quantity;
11    stockOnLoan += quantity;
12    return true;
13 }
14
15 public boolean receive(int quantity) {
16     if (quantity <= 0 || quantity > stockOnLoan) return false
17         ;
18    stockAvailable += quantity;
19    stockOnLoan -= quantity;
20    return true;
21 }
```

Listing 1: Métodos de mutación de stock con validaciones en InventoryItem

```
1 public interface InventoryIndex {
2     void put(Integer key, InventoryItem value);
3     InventoryItem get(Integer key);
4     void delete(Integer key);
5     boolean contains(Integer key);
6     Iterable<Integer> keys();
7     int size();
8     int height();
9 }
```

Listing 2: Contrato unificado provisto por la interfaz InventoryIndex

2.4. Generador de Operaciones y Decisiones de Diseño

Para realizar una comparación experimental válida, ambas estructuras debían recibir exactamente las mismas operaciones en el mismo orden. Para asegurarlo, la clase `DataGenerator` amarró el azar a una semilla utilizando `StdRandom.setSeed(seed)`.

Una decisión de diseño clave para construir el generador fue apoyarse internamente en un `HashSet<Integer>` llamado `presentKeys`. Esto facilitó la sincronización lógica de las operaciones generadas al azar: si salía una instrucción `PURCHASE` para una clave que no estaba en el set de componentes registrados, el programa instanciaba un `InventoryItem` nuevo. Por el contrario, si la clave ya existía, simplemente se emitía una operación de recarga de stock, enviando `null` en el campo del ítem para no sobrescribir la referencia original.

```
1  if (!presentKeys.contains(key)) {
2      // Ingreso de un componente nuevo
3      InventoryItem newItem = generateItem(key);
4      ops.add(new InventoryOperation(OperationType.PURCHASE,
5          key, quantity, newItem));
6      presentKeys.add(key);
7  } else {
8      // Reposición de stock de un componente existente
9      ops.add(new InventoryOperation(OperationType.PURCHASE,
10         key, quantity, null));
11 }
```

Listing 3: Lógica de control de duplicados mediante HashSet

3. Etapas 2 y 3: Análisis Empírico

3.1. Estructura del software experimental

Las mediciones se estructuraron usando el método de *Doubling Ratio*. Se probaron volúmenes de datos donde $m = 2^t$, empezando desde $t = 12$ ($m = 4,096$) y finalizando en $t = 19$ ($m = 524,288$). En total, se ejecutaron 30 repeticiones por cada tamaño para poder obtener promedios estadísticos representativos (480 iteraciones totales).

El tiempo de ejecución se midió de forma exclusiva con **StopwatchCPU**. Para asegurar que la prueba fuera lo más limpia posible, el cronómetro se aisló de procesos secundarios. Tareas como la generación de los arreglos, las comprobaciones de consistencia posteriores y la escritura de los CSV quedaron fuera del bloque de medición.

3.2. Desafíos enfrentados

El diseño de este experimento lidió con dos factores propios de la Máquina Virtual de Java (JVM) que afectaban las mediciones iniciales:

El impacto del compilador JIT: En los tamaños más pequeños ($m = 4,096$), la JVM interpreta el código antes de que el compilador *Just-In-Time* (JIT) lo traduzca y lo optimice a código nativo. Esto genera tiempos de ejecución más altos en las primeras vueltas del bucle. La solución fue aplicar una etapa de calentamiento o *Warm-up Phase*, procesando 10.000 operaciones en vacío para darle tiempo al compilador de optimizar las rutinas.

El Recolector de Basura (Garbage Collector): En las pruebas de estrés más pesadas ($m = 524,288$), instanciar cientos de miles de objetos **InventoryOperation** saturaba rápidamente la memoria Heap. Si el *Garbage Collector* (GC) entraba en acción mientras se cronometraba una estructura, la métrica final registraba milisegundos que no correspondían al rendimiento real de la búsqueda. Para evitarlo, se ejecutó una llamada manual a **System.gc()** justo antes de arrancar cada **StopwatchCPU**.

3.3. Mediciones empíricas

Al revisar los promedios obtenidos en el Cuadro 1, se identifica un patrón de rendimiento muy marcado. En las cargas pequeñas los tiempos están prácticamente a la par. Sin embargo, al alcanzar el límite de estrés ($m = 524,288$), el tiempo medio del BST convencional se vuelve más alto y registra una mayor dispersión frente a los resultados consistentes del **RedBlackBST**.

Cuadro 1: Tiempos promedio de ejecución y Alturas del árbol (30 instancias)				
m	Tiempo BST (s)	Tiempo RB (s)	Altura BST	Altura RB
4.096	0,0039	0,0041	24,0	13,0
8.192	0,0056	0,0059	28,0	14,0
16.384	0,0073	0,0069	30,0	16,0
32.768	0,0221	0,0165	31,0	17,5
65.536	0,0445	0,0390	34,5	18,5
131.072	0,0655	0,0581	36,0	20,5
262.144	0,1828	0,1546	38,5	22,0
524.288	0,4814	0,4194	44,0	23,5

La principal causa de esta diferencia de rendimiento se observa en la segunda métrica: **la altura física del árbol**. Mientras que el RedBlackBST logró procesar el medio millón de operaciones limitando su altura promedio a 23,5 niveles, el BST clásico perdió su simetría a lo largo de las inserciones y eliminaciones, llegando a alcanzar una profundidad de 44 niveles.

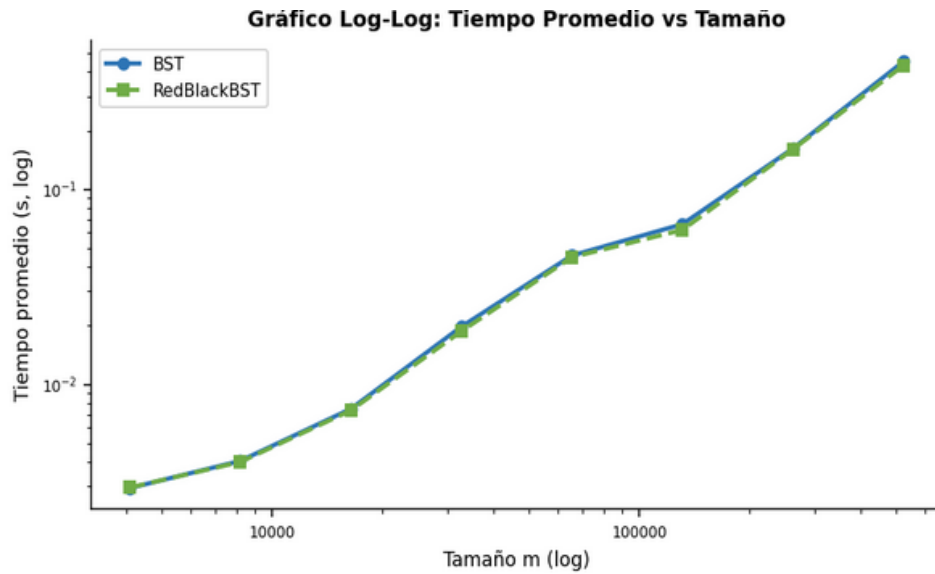


Figura 1: Gráfico log-log: Tiempo de ejecución promedio vs. Tamaño de la prueba (m).

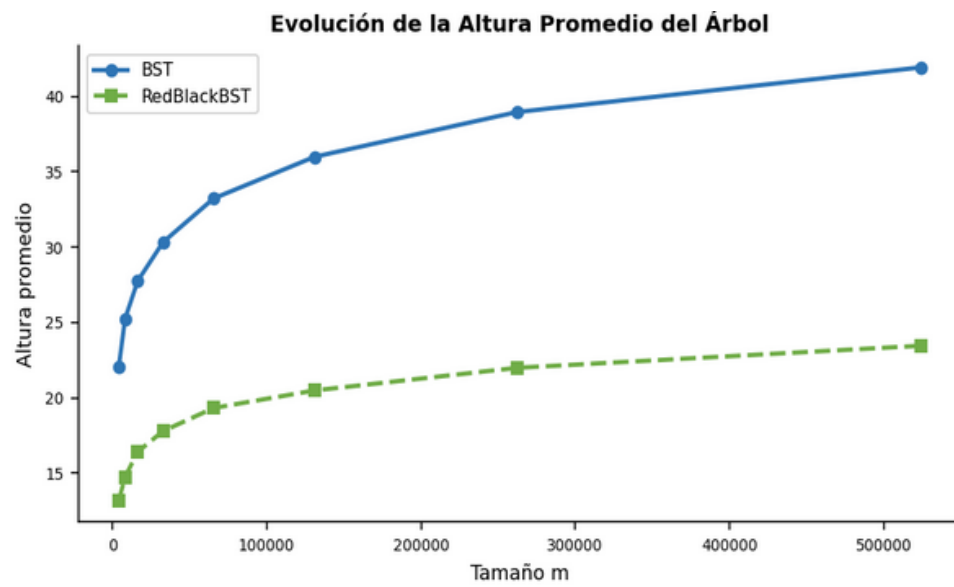


Figura 2: Evolución promedio de la altura física de los árboles.

4. Etapa 4: Análisis Teórico Big-O

4.1. Tabla de símbolos

Las operaciones propias del sistema de inventario (registrar nuevos componentes, actualizar existencias y darlos de baja) se traducen de forma directa a las tres funciones principales de una Tabla de Símbolos: `put`, `get` y `delete`.

4.2. BST — Caso promedio $O(\log n)$, Peor caso $O(n)$

En un árbol binario de búsqueda, la ubicación de un nodo depende netamente de comparar las claves en su descenso desde la raíz. Por lo tanto, el costo de sus operaciones depende de qué tan profundo deba viajar el algoritmo (su altura h):

- **Caso Promedio:** Si las claves ingresan en un orden aleatorio uniforme, la altura esperada crece de forma moderada, resultando en costos de la magnitud $O(\log n)$ para sus métodos.
- **Peor Caso:** Si las claves se ingresan en orden, o si el árbol acumula desbalances producto del método asimétrico de borrado de Hibbard, la estructura degenera en algo similar a una lista enlazada. Su altura pasa a ser $h = n$, provocando que el rendimiento colapse a un tiempo ineficiente de $O(n)$.

4.3. RedBlackBST — Balanceo $O(\log n)$ garantizado

La versión programada por Sedgwick y Wayne del árbol rojo-negro emula internamente un árbol 2-3, utilizando punteros de color rojo para representar las uniones de los nodos lógicos de tres conexiones. Sus reglas de inserción y rotación aseguran que: (1) los enlaces rojos siempre apuntan hacia la izquierda; (2) no pueden existir dos enlaces rojos consecutivos descendiendo; (3) existe un balance perfecto de los enlaces negros en todos los caminos hasta las hojas nulas.

Esta configuración matemática garantiza que la longitud del camino más largo en el árbol nunca será superior al doble de la del camino más corto. Gracias a esto, la altura total queda rígidamente acotada a $2\log_2(n+1)$, asegurando una complejidad asintótica de $O(\log n)$ en el peor caso posible para los tres métodos principales (`put`, `get` y `delete`).

5. Etapa 5: Comparación Teoría vs. Práctica

Las métricas obtenidas empíricamente en la plataforma concuerdan a la perfección con lo descrito en el análisis teórico.

Análisis de Ratios Empíricos:

La teoría señala que si ejecutamos m transacciones y cada una exige un costo individual de búsqueda en $O(\log m)$, la aplicación total operará en $O(m \log m)$. Utilizando el factor de crecimiento a través del cálculo del ratio, observamos el siguiente límite:

$$\text{ratio} = \frac{T(2m)}{T(m)} \approx \frac{2m \log_2(2m)}{m \log_2(m)} \rightarrow 2 \implies \log_2(\text{ratio}) \rightarrow 1,0$$

Al analizar las columnas generadas en la pestaña Resumen del archivo Excel, los valores calculados para $\log_2(\text{ratio})$ se estabilizan muy de cerca al valor de 1,03, demostrando en la práctica la validez de la cota teórica.

El impacto de la altura sobre la CPU:

Un aspecto destacable de las mediciones es que la ventaja de CPU a favor del RedBlackBST en la prueba mayor de $m = 524,288$ ($\sim 12,8\%$ más veloz) es porcentualmente menor que la diferencia física de alturas registrada (el BST resultó un 87% más profundo). La explicación detrás de esto recae en el hecho de que la mayoría de las búsquedas al azar logran coincidencias en los estratos medios o altos de la estructura (*internal path length*). Ese costo promedio logra compensar parcialmente el sobretiempo que gasta la CPU al tener que visitar el fondo de la rama más profunda del árbol desbalanceado.

6. Conclusiones

Tras evaluar el desempeño en código de ambas estructuras frente a una simulación de alto flujo transaccional, es posible llegar a las siguientes conclusiones:

1. **Mejor desempeño general:** El RedBlackBST cuenta con un mejor rendimiento promedio para tareas a gran escala. Si bien mantener sus propiedades involucra un mínimo de operaciones extras para rotar y colorear nodos, este costo computacional es despreciable frente a las ventajas de mantener los recorridos de búsqueda cortos a lo largo del tiempo.
2. **Tiempos de respuesta más estables:** El RedBlackBST obtuvo desviaciones estándar más bajas en los tamaños grandes. Esto se traduce en latencias mucho más predecibles en cada consulta, un factor crítico en bases de datos reales.
3. **Mantenimiento de la estructura:** A pesar de enfrentarse a un 10 % de operaciones de borrado (DISPOSE) entremezcladas con nuevas creaciones, el RedBlackBST preservó su altura logarítmica sin deformarse, demostrando el éxito de sus métodos de rebalanceo proactivo.
4. **Aplicación en un entorno de producción:** Intentar gestionar un inventario concurrente y crítico utilizando un BST estándar expone a los servicios a caídas eventuales de rendimiento. La implementación del RedBlackBST es la opción funcional y requerida para arquitecturas de software robustas.
5. **El retorno de inversión del balanceo:** Se logró comprobar experimentalmente que el sobreesfuerzo que realiza el algoritmo al insertar datos (rotar y balancear punteros) es altamente rentable para la CPU, ya que previene por completo la degradación asintótica del TDA a futuro.
6. **Teoría versus Práctica:** Aunque la notación Big-O es la base principal para comprender cómo escalan las estructuras, realizar el experimento empírico fue fundamental para notar la influencia directa de la máquina, validando la importancia de tener en cuenta el impacto de agentes como la recolección de basura o el JIT Compiler al momento de desarrollar algoritmos.

Referencias

1. Guía de Laboratorio 4: Árboles de búsqueda. ESTRUCTURAS DE DATOS Y ALGORITMOS 2026-1. Universidad Diego Portales.
2. Sedgewick, R. & Wayne, K. (2011). *Algorithms, 4th Edition*. Addison-Wesley.
<https://algs4.cs.princeton.edu/>